THE YLANG BOOK

A practical reference for YLang 2.0.0

Stable core language, compiler workflow, standard library, and experimental extensions

VERSION 2.0.0 STABLE

Standalone PDF edition - generated from repository documentation

The default compiler generates C and uses GCC or Clang. Experimental extensions are labelled separately and do not share the stable-core compatibility guarantee.

Contents

Introduction . . . 3
Install YLang . . . 4
Your First Program . . . 6
The Compiler Workflow . . . 7
Language Basics . . . 8
Types and Explicit Conversions . . . 9
Functions and Modules . . . 10
The Standard Library . . . 11
GUI and Game2D . . . 12
Platform Support . . . 13
Stable Core Language Specification . . . 14
Compiler Architecture . . . 20
Structs (Experimental) . . . 22
Enums and Matching (Experimental) . . . 23
GUI API (Experimental) . . . 24
Game2D Library (Experimental) . . . 27
NASM Backend Roadmap . . . 30
Windows Setup (Experimental) . . . 32
YSPM Linux Packages . . . 34
Compiler Error Codes . . . 36
YLang 2.0.0 Stable Release Notes . . . 37
Option and Result Design Proposal . . . 39

Introduction

Source: docs/online-book/src/introduction.md

YLang is a small compiled programming language with explicit types and actionable compiler diagnostics. Its compiler is written in C17. The default build path checks a `.yl` program, emits C, and invokes GCC or Clang to create a native executable.

This book combines a progressive tutorial with the project's detailed reference documents. The tutorial chapters help you get a program running; the reference chapters describe the stable core, diagnostics, platform requirements, and the exact limits of experimental extensions.

What YLang is—and is not

YLang is compiled, not an interpreter. The compiler checks syntax, names, initialization, and types before generating and compiling the program. The C backend remains the broad-coverage default. An experimental NASM backend supports only a restricted Linux x86-64 subset.

YLang does not provide every feature of a mature systems language. `Option<T>` and `Result<T, E>` are not implemented. Structs, enums, imports, GUI/Game2D APIs, the NASM backend, and native Windows support remain experimental extensions. YLang makes no Rust-level memory-safety claim.

Stable and experimental material

Throughout the book, the stable 2.0.0 core is distinguished from experimental extensions. Platform and feature pages state which paths are tested and which remain experimental. Do not infer that a syntax feature works in every backend just because the parser accepts it; consult the relevant reference chapter.

Continue to install YLang (getting-started.md), or open the complete language specification (reference/language-spec.md) when you need an exact syntax and behavior reference.

Install YLang

Source: docs/online-book/src/getting-started.md

This chapter builds YLang from source. It is the most direct way to use the compiler and run the regression tests.

Requirements

- Linux, macOS, or the experimental 64-bit Windows path described in Platform Support (platform-support.md).
- GCC or Clang with C17 support.
- GNU Make (`make` on Linux; `gmake` on macOS).

GUI and Game2D programs also need SDL2 development files and `pkg-config`. Those packages are optional for ordinary command-line programs.

Linux

On Fedora:

```
sudo dnf install gcc clang make
```

On Debian or Ubuntu:

```
sudo apt install build-essential clang make
```

Then clone and build the project:

```
git clone https://github.com/Yassine-Jemi01/ylang.git
cd ylang
make
make test
```

The compiler is written to `build/ylang`.

macOS

Install Apple's command-line tools and the build dependencies:

```
xcode-select --install
brew install make llvm
```

Use GNU Make as `gmake` for the build and test commands:

```
gmake
gmake test
```

For GUI/Game2D builds, also install SDL2 and `pkg-config`:

```
brew install sdl2 pkg-config
```

Windows

The initial native Windows target uses 64-bit MinGW-w64 from an MSYS2 MINGW64 shell. MSVC is not the supported toolchain for this milestone. Follow the Windows setup guide (reference/WINDOWS.md) for exact packages and limitations.

Linux package option

YLang also provides an optional `.yspkg` package for compatible Linux x86-64 systems using YSPM. YSPM is the system package manager; it is not a project-local YLang module manager. See YSPM Linux Packages (reference/YSPM.md).

Next

Continue to Your First Program (hello-world.md) to check, compile, and run a `.yl` file.

Your First Program

Source: docs/online-book/src/hello-world.md

Create a file named `hello.yl`:

```
function main() -> int {
    let const string name = "YLang";
    print("Hello from", name);
    print("The answer is", 40 + 2);
    return 0;
}
```

The program defines the `main` function, creates a constant string, prints two lines, and returns an integer exit status.

Check, build, and run

From the repository root:

```
./build/ylang check hello.yl
./build/ylang build hello.yl -o hello
./hello
```

The first command checks the source without producing an executable. The second uses the default C backend and invokes a C compiler to build the native program. The final command runs it.

On Windows, the compiler executable is normally `build/ylang.exe`; the output program should use a Windows executable name such as `hello.exe`.

What if it fails?

Read the complete diagnostic, including its error code. Check punctuation, declared types, and the compiler command before changing the program. The Compiler Error Codes (reference/error-codes.md) page explains stable diagnostic identifiers.

Try one change

Change the value of `name`, or replace `40 + 2` with a different integer expression. Run the same check/build/run sequence again. The point is to build a habit: check the source first, then compile it, then run the output.

The Compiler Workflow

Source: docs/online-book/src/compiler-workflow.md

The everyday workflow has three steps: check, build, and run.

Check before building

```
./build/ylang check hello.yl
```

Use `check` while editing. It reports syntax, name, initialization, and type errors without compiling the source to a final executable.

Build an executable

```
./build/ylang build hello.yl -o hello
```

The default backend emits C and calls GCC or Clang. Use `--cc clang` to select Clang explicitly:

```
./build/ylang build hello.yl -o hello --cc clang
```

The C compiler is part of this route: YLang does not yet have a complete general-purpose native backend independent of GCC/Clang.

Inspect generated code

```
./build/ylang emit-c hello.yl -o build/hello.c
```

Generated code can help you understand the compilation pipeline or investigate a compiler issue. The Compiler Architecture (reference/architecture.md) chapter explains the major stages.

Experimental NASM path

On supported Linux x86-64 systems, the compiler can emit NASM assembly or build a program from the restricted NASM subset:

```
./build/ylang emit-asm examples/your-program.yl -o build/your-program.asm
./build/ylang build-asm examples/your-program.yl -o build/your-program
```

NASM is experimental. It does not support every construct accepted by the C backend, and unsupported combinations should be rejected explicitly. See the Backend Roadmap (reference/BACKEND-ROADMAP.md) before choosing it for a project.

Language Basics

Source: docs/online-book/src/language-basics.md

YLang uses explicit type names in variable declarations and function signatures. Most programs define a `main` function that returns an integer status.

```
function add(int left, int right) -> int {
    return left + right;
}

function main() -> int {
    let int total = add(20, 22);
    print(total);
    return 0;
}
```

The exact supported syntax is defined by the Language Specification (reference/language-spec.md). Use that reference whenever an example and the compiler disagree.

Variables

A mutable local uses a declaration such as:

```
let int score = 10;
score = score + 1;
```

A constant declaration uses `const`:

```
let const string language = "YLang";
```

YLang checks assignments and uses of values against their declared types. A value must be initialized before it is read, and a constant cannot be reassigned.

Control flow

Conditional statements and loops let a function choose and repeat work. Keep the expression types explicit and run `ylang check` after each change. Consult the specification for the complete grammar supported by this beta; do not assume that syntax from C, Python, or Rust is accepted just because it looks familiar.

User-defined data

The 2.0 beta adds experimental structs and unit enums. These features are useful for small programs, but their limits and backend support are not the same as Rust's. Continue to the Structs (reference/STRUCTS.md) and Enums and Matching (reference/ENUMS.md) chapters for their current contracts.

Types and Explicit Conversions

Source: docs/online-book/src/types.md

YLang declarations make types visible. The language does **not** silently convert arbitrary values between types.

The common built-in types include `int`, `float`, `bool`, `char`, and `string`. Use the supported type and expression forms in the Language Specification (reference/language-spec.md).

Convert intentionally

Use explicit conversion functions when you need a different representation:

- `to_float(integer)` converts an integer to a floating-point value.
- `to_int(float)` truncates toward zero and reports a runtime error for non-finite or out-of-range values.
- `to_string(value)` formats supported scalar values as a string.
- `string.is_int(text)` checks whether the full text is a valid signed integer representation.
- `string.parse_int(text)` parses integer text with failure handling.

These functions make a conversion visible at the call site instead of hiding it inside an assignment or expression. See the Language Specification (reference/language-spec.md) for the precise behavior and supported cases.

Type errors are useful

When a value has the wrong type, change the declaration, correct the expression, or use a documented explicit conversion. Avoid adding a conversion just to silence an error unless it preserves the intended meaning.

Not implemented in this beta

`Option<T>` and `Result<T, E>` have design notes but are not implemented language constructs. Refer to Option and Result Design (reference/OPTION-RESULT-DESIGN.md) for the proposal; do not use them as current syntax.

Functions and Modules

Source: docs/online-book/src/modules.md

Functions provide named units of behavior; imports let a source file use selected declarations from another `.yl` file.

A local import

Given a sibling file named `math_tools.yl`:

```
function answer() -> int {
    return 42;
}
```

A second file can import that function:

```
from "./math_tools.yl" import answer;

function main() -> int {
    print(answer());
    return 0;
}
```

Relative paths are resolved from the importing source file. In this beta, imports are experimental and do not imply a central package registry or dependency solver. Import aliases and qualified module namespaces are not part of the current contract.

Organize one module at a time

Keep each module focused, import only the names the file needs, and run `check` on the entry-point file after changing the import graph. The compiler checks imported symbols and reports missing files or names; circular imports are rejected.

For the exact grammar, visibility rules, and restrictions, read the Module System (reference/MODULE-SYSTEM-SPEC.md) chapter.

The Standard Library

Source: docs/online-book/src/standard-library.md

YLang's standard library provides small built-in namespaces for common tasks. The available surface depends on the current documented language subset; use the reference pages rather than assuming that a similarly named function exists in another language.

The current documentation covers namespaces such as:

- `math` for supported mathematical operations.
- `string` for string inspection and explicit parsing/conversion helpers.
- `io` for console and file-related operations.
- `path` for path-related checks.
- `image` for supported image-related operations.

The Language Specification (reference/language-spec.md) documents the stable language subset and conversion rules. The Compiler Error Codes (reference/error-codes.md) page includes diagnostics for unknown standard-library namespaces. GUI and game helpers are optional and have their own SDL2 requirements; start with GUI and Game2D (gui-game2d.md).

Do not assume every standard-library call is supported by the experimental NASM backend. The C backend remains the broad-coverage default.

GUI and Game2D

Source: docs/online-book/src/gui-game2d.md

The optional GUI runtime uses SDL2. The Game2D helper module is a small, experimental layer for learning and simple 2D prototypes—not a general-purpose game engine.

It provides helpers for window lifecycle, event polling, keyboard and mouse input, elapsed frame time, rectangle drawing, collision checks, and presentation. The repository includes a playable Dodge example.

Build the example

Install SDL2 development files and `pkg-config` for your platform, then from the repository root run:

```
make game2d-example
```

This builds the example as `build/dodge` on Linux and macOS, or `build/dodge.exe` on Windows. Run it from a graphical desktop session:

```
./build/dodge
```

On Windows, run `build/dodge.exe` from the appropriate MSYS2 environment.

Understand the limitations

Headless CI verifies type-checking, compilation, linking, and non-window helper behavior. It does not verify that a live window behaves correctly on every desktop or driver. Test the game on a real graphical session before relying on it.

The library does not yet provide sprites, advanced animation, audio, a scene system, or a complete physics engine. See the Game2D Library Reference (reference/GAME2D.md) and the GUI API Contract (reference/GUI-API-SPEC.md).

Platform Support

Source: docs/online-book/src/platform-support.md

YLang has different support levels on each platform. Installation instructions do not imply the same backend coverage everywhere.

Linux

Linux is a primary development platform for the C backend and the experimental NASM x86-64 subset. A compatible Linux x86-64 system can also install the optional YSPM package; see YSPM Linux Packages (reference/YSPM.md).

macOS

macOS is tested with Clang and GNU Make. Use `gmake` rather than the system BSD `make`. GUI/Game2D programs need SDL2 development files and `pkg-config`.

Windows

The initial native path targets 64-bit Windows with MinGW-w64 from MSYS2 MINGW64. CI tests compilation, a native executable, relative imports, standard input, and SDL2 build/link. MSVC is not supported by this first milestone, and full Windows feature parity is not claimed.

Read the Windows Setup (reference/WINDOWS.md) guide for toolchain commands and current restrictions.

Stable Core Language Specification

Source: docs/language-spec.md

This document defines the stable core language contract for YLang 2.0.0. Named imports, structs, enums, GUI/Game2D APIs, the NASM backend, native Windows support, and YSPM packaging are documented separately and remain experimental; their behavior is not covered by the stable-core compatibility guarantee. Syntax not listed here is not part of the stable core contract.

Explicit conversions

YLang does not implicitly convert values between types. Use explicit conversion functions:

- `to_float(integer)` converts a signed 64-bit integer to a double-precision float.
- `to_int(float)` truncates toward zero and terminates with a runtime error if the input is non-finite or outside the signed 64-bit range.
- `to_string(value)` supports `int`, `float`, `bool`, `char`, and `string`. A string input is returned unchanged.
- `string.is_int(text)` returns whether the entire string is a valid base-10 signed 64-bit integer.
- `string.parse_int(text)` converts a valid integer string; invalid or out-of-range input produces a runtime error. Check with `string.is_int` first when parsing untrusted input.

1. Compilation model

YLang is a compiled language. `ylang check` parses and performs semantic checks. `ylang build` generates C and invokes GCC or Clang to produce a native executable. `ylang emit-c` writes C for inspection. There is no interpreter in this release.

2. Source form and statements

Statements end with `;`, blocks use `{` and `}`, and comments use `//` through the end of a line or `/* ... */` across lines. Block comments cannot be nested and must be closed. Keywords are lowercase and case-sensitive. Source files are text and embedded NUL bytes are rejected before lexing. Each source file passed to the CLI is compiled as one program. A program must define `function main() -> int` or `function main() -> void`, with no parameters.

3. Variables and constants

```
let int age;
let int score = 100;
let string name = "YLang";
let bool enabled = true;
let const int LIMIT = 100;
```

- Mutable declaration: `let type name;` or `let type name = expression;`.
- Constant declaration: `let const type name = expression;`. Constants must be initialized and cannot be reassigned.
- A local variable must be assigned on every path before it is read. The checker rejects a read it cannot prove initialized.
- Global variables without explicit initializers receive a type-appropriate zero/empty default. Explicit initialization is preferred.
- A declaration can shadow a name in an outer block, but duplicate names within one scope are errors.
- Assignment is a statement, not a general nested expression.

4. Types

Type	Meaning
`int`	Signed 64-bit integer
`float`	64-bit floating-point value
`bool`	`true` or `false`
`char`	One byte; ASCII-oriented
`string`	Immutable string data represented as a C string by the current backend
`void`	Function return type only; cannot be used as a variable type

No implicit numeric conversions are performed. Arithmetic operands must have compatible matching types. `%` is available only for `int`. Conditions must have type `bool`. Strings support equality and lexicographic ordering comparisons. String values are immutable.

User-defined enums (experimental)

Unit enums are supported as named, distinct types:

```
enum Direction { North, East, South, West }

function is_east(Direction direction) -> bool {
    return direction == Direction.East;
}

function main() -> int {
    let Direction facing = Direction.North;
    if (is_east(Direction.East)) { print("east"); }
    if (facing == Direction.North) { print("north"); }
    return 0;
}
```

Declare an enum before using its type. Refer to variants with `EnumName.Variant`. Values of different enum types cannot be mixed, and a variant can be passed to or returned from a function with the same enum type. Variants currently have no payloads. A statement `match (value) { EnumName.Variant -> { ... } }` must contain exactly one arm for every variant; missing and duplicate arms are errors. For this milestone, the matched expression must not contain calls or assignments. Enum formatting with `print`, importing enum types across module boundaries, payload variants, and explicit numeric discriminants are not implemented yet. The C backend and the experimental NASM backend represent these unit variants as integer values while semantic analysis preserves their distinct YLang types.

5. Literals and strings

```
let int n = 42;
let float ratio = 1.25;
let bool ready = true;
let char initial = 'Y';
let string message = "Hello\n";
let string greeting = f"Hello {message}";
```

`char` is one byte, not a Unicode scalar. Integer literals support decimal notation and hexadecimal notation such as `0xFF`; floating-point literals support decimal notation and scientific notation such as `1e3` or `2.5e-4`. A string literal must be terminated. F-strings begin with `f"` and interpolate expressions inside `{}`. Supported interpolations include variables and operators. Function calls and assignments are not allowed inside interpolation expressions; compute the value in a preceding statement. Literal braces can be escaped as documented by the lexer (`{{`, `}}`, `\{`, and `\}`).

6. Output

`print(expression, ...)` writes values separated by one space and appends a newline:

```
print("answer:", 42);
print(f"Count: {n}");
```

Supported printable values are `int`, `float`, `bool`, `char`, and `string`. `print` is a statement in this release.

7. Expressions and operators

- Unary: `-`, `not`
- Arithmetic: `+`, `-`, `*`, `/`, `%`
- Comparison: `==`, `!=`, `<`, `<=`, `>`, `>=`
- Boolean: `and`, `or`
- Assignment: `=` and `+=` as statements; postfix `++` increments a mutable numeric variable by one
- Grouping: `(expression)`
- Calls: `name(argument, ...)`

Operator precedence is conventional: unary operators, multiplication/division/modulo, addition/subtraction, comparisons/equality, `and`, then `or`. Parentheses may be used to make grouping explicit.

Signed integer overflow, integer division/modulo by zero, and floating-point division by zero are handled by generated runtime helpers and terminate the program with a runtime-error message and status 70. This does not mean every possible C-level issue is guarded or that the language is memory-safe.

Loops

Three loop forms are supported:

```
loop() {
    // Repeat until break.
    break;
}

while (count < limit) {
    count += 1; or count++;
}

for (let int i = 0; i < limit; i = i + 1) {
    print(i);
}
```

- `loop()` is an unconditional loop.
- `while (condition)` repeats while a boolean condition is true.
- `for (initializer; condition; increment)` uses C-style clauses. The initializer may be a `let` declaration or expression; condition and increment may be omitted.
- `break;` exits the innermost loop and `continue;` starts its next iteration.
- A `for` initializer declaration is scoped to the loop and its body.

Fixed-size arrays

YLang supports fixed-size arrays with an explicit element type and an initializer. The length is inferred from the initializer. Arrays may be local or global, must contain at least one element, and all elements must have the same type.

```
let int scores[] = [10, 20, 30];

function main() -> int {
    scores[1] = 42;
    print(scores[0], scores[1], scores[2]);
    return 0;
}
```

- Indexing starts at zero.
- Array indices must be `int`; every access performs a runtime bounds check and exits with status 70 on an invalid index.
- Element assignment is supported, such as `scores[1] = 42;`.

- Whole-array assignment, returning arrays, nested arrays, and `const` arrays are not supported yet. Array parameters use `type name[]`; the generated native ABI passes a pointer and a separate length value.
- Arrays must be initialized at declaration; their size cannot change at runtime.

Standard string and input library

YLang includes byte-oriented string helpers and a line reader for simple command-line programs:

```
let string name = io.read_line();
let string greeting = string.concat("Hello, ", name);
print(greeting);
print(string.length(name));
print(string.contains(name, "lang"));
print(string.starts_with(name, "Y"));
print(string.ends_with(name, "n"));
```

- `string.length(value)` returns the number of bytes, not Unicode characters.
- `string.contains(value, needle)`, `string.starts_with(value, prefix)`, and `string.ends_with(value, suffix)` return `bool`.
- `string.concat(left, right)` returns a new string.
- `string.replace(value, needle, replacement)` returns a new string with every non-overlapping occurrence of `needle` replaced. An empty `needle` leaves the original string unchanged.
- `string.upper(value)` and `string.lower(value)` return case-converted copies using the C locale's byte-oriented character mapping; they are not Unicode-aware.
- Strings support lexicographic ordering with `<`, `<=`, `>`, and `>=`, using bytewise C-string ordering. Equality remains case-sensitive.
- The generated runtime tracks allocated strings until process exit, so repeated concatenation or case conversion in a long-running loop can increase memory use.
- `io.read_line()` reads one line from standard input and removes its trailing newline. At end-of-file it returns an empty string.
- `io.read_file(path)` reads an entire UTF-8/ASCII-compatible text file into a string. A file-open/read failure or embedded NUL byte terminates the program with a runtime error; binary files are not supported by the string API.
- `io.write_file(path, content)` overwrites or creates a text file and returns `true` on success or `false` if it cannot open/write the file. It does not create missing parent directories.
- File paths are interpreted relative to the process working directory. File I/O is synchronous and currently has no structured error/exception type.\n- `path.exists(path)` returns whether a path can be inspected with `stat`; it may return true for files, directories, or other filesystem objects.\n- `path.basename(path)` returns the final path component, and `path.extension(path)` returns the suffix after the final dot (without the dot); a leading dot alone is not treated as an extension. These helpers use POSIX path separators.\n- `image.open(path)` launches the platform default opener (`xdg-open` on Linux, `open` on macOS) without invoking a shell and returns whether the launcher exits successfully. It requires a desktop session and a registered handler; it opens the file in an external application and does not decode or draw the image in a YLang window.

Path and desktop integration\n\n``ylang\nlet string file = io.read_line();\nif (path.exists(file)) {\n print(path.basename(file), path.extension(file));\n let bool opened = image.open(file);\n if (not opened) { print("Could not launch the default image/file viewer."); }\n}\n`\n\nThe path namespace exposes exists, basename, and extension. The image.open(path)` convenience API starts the operating system's default file handler without passing the path through a shell, avoiding shell-metacharacter interpretation. This is OS integration rather than a built-in image decoder or a custom GUI image widget; native GUI bindings and in-process image decoding remain future work. These APIs currently target the POSIX systems supported by YLang.\n\n## Standard math library

YLang provides a built-in `math` namespace for common floating-point operations. These calls are checked by the compiler and emitted as native C math-library calls.

```
let float root = math.sqrt(81.0);
let float power = math.pow(2.0, 8.0);
let float angle = math.sin(0.0);
let float bounded = math.clamp(12.0, 0.0, 10.0);
```

Available functions (all arguments must be `float`, and all return `float`):

- Unary: `sqrt`, `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh`, `exp`, `log`, `log10`, `floor`, `ceil`, `round`, `abs`.
- Binary: `pow`, `atan2`, `min`, `max`, `hypot`.
- Three arguments: `clamp(value, low, high)`; a lower bound greater than the upper bound triggers a runtime error.

YLang does not implicitly convert integers to floats, so use float literals such as `9.0`. Domain errors and non-finite results follow the platform C math library behavior.

8. Conditions and loops

```
if (age >= 18) {
    print("Adult");
} else if (age >= 13) {
    print("Teenager");
} else {
    print("Child");
}

loop() {
    if (done) {
        break;
    }
    continue;
}
```

Conditions must be boolean. `loop()` repeats indefinitely until `break` executes. `while` repeats while its condition is true, and `for` supports initializer, condition, and increment clauses. `continue` skips to the next iteration. `break` and `continue` are only valid inside a loop.

9. Functions

```
function add(int left, int right) -> int {
    return left + right;
}

function main() -> int {
    print(add(20, 22));
    return 0;
}
```

Parameter and return types are explicit. Function overloading is not supported. A non-void function must return a value on supported control-flow paths. `main` has no parameters and returns `int` or `void`.

10. Modules and imports

A source file can import named functions or global variables from another `.yl` file:

```
from "./system_data.yl" import system_data, system_prefix;

function main() -> int {
    print(system_data(), system_prefix);
    return 0;
}
```

Import paths are resolved relative to the file containing the import, not the

process working directory. Imports are transitive: an imported module may import

names from another module. Every name used from another module must be imported

explicitly. Functions and globals declared in a module are visible inside that

module, but do not become visible to importers unless their names are imported.

Different modules may define private helpers with the same name.

The compiler rejects unknown imported names, conflicting or repeated imports,

imports of the program entry point, missing module files, and circular imports.

The root source file must define `main()`; an imported module cannot define it.

Imports are currently supported by the default C backend. The experimental NASM

backend rejects multi-file imports explicitly until module-aware assembly

lowering is implemented.

11. Diagnostics and tooling

`check` reports parse and semantic/type diagnostics without invoking a native compiler. Diagnostics include an error code, file/line/column, source excerpt, caret, and a hint when one is available. `build` does not produce a successful executable if YLang diagnostics contain errors.

`fix input.yl -o output.yl` currently implements one high-confidence source transformation: changing a `pritn(...)` call typo to `print(...)` when no function named `pritn` is declared. It requires an output path and does not intentionally modify comment/string text. It is not a general repair engine. Always run `check` after applying a fix.

12. Unsupported features and implementation limits

The following are not supported by the current language subset: structs/classes/OOP, payload enums, generic `Option<T>`/`Result<T, E>`, exception syntax (`try`/`catch`), generics, raw pointers/references, returning arrays, nested/const arrays, and a dedicated LLVM/native backend. `class`, `try`, and `catch` may be tokenized as reserved words but are not valid executable constructs.

The compiler generates C, so native code inherits ordinary process privileges and is not sandboxed. Some formatted string values are stored until process exit; creating many such values in a long-running loop can increase memory consumption. The language does not define ownership/borrowing or a garbage collector in this release. Do not assume Rust-like memory-safety guarantees.

Compiler Architecture

Source: docs/architecture.md

YLang 1.0.0 uses a conventional multi-stage compiler pipeline:

1. **Lexer (`src/lexer.c`)** — turns source text into positioned tokens.
2. **Parser (`src/parser.c`)** — builds the program representation and emits syntax diagnostics.
3. **Semantic analysis (`src/sema.c`)** — resolves names, validates initialization and constant assignment, and checks types.
4. **Module loader (`src/module.c`)** — recursively loads imported files, detects cycles, resolves named imports, and merges module ASTs while retaining per-module visibility.
5. **C code generator (`src/codegen.c`)** — writes C for the checked program, including runtime helpers for supported checked arithmetic operations.
6. **Driver (`src/driver.c`)** — orchestrates module loading and checking, writes generated C, and invokes GCC or Clang without going through a shell.
7. **CLI (`src/main.c`)** — parses user commands and their options.

The internal compiler model is declared in `src/internal.h`; the public-facing compiler and token declarations are in `include/ylang/`.

Experimental NASM backend (Linux x86-64)

The optional emit-asm command writes NASM syntax for a deliberately limited

subset of YLang. Unlike the default C backend, the generated program uses the

Linux x86-64 system-call ABI for output and process exit, so a generated

program can be assembled with NASM and linked directly with ld without

GCC/Clang or a C runtime:

./build/ylang emit-asm tests/asm-basic.yl -o build/asm-basic.asm

nasm -f elf64 build/asm-basic.asm -o build/asm-basic.o

ld -o build/asm-basic build/asm-basic.o

./build/asm-basic

For the same steps in one command, use build-asm:

./build/ylang build-asm tests/asm-basic.yl -o build/asm-basic

./build/asm-basic

The initial subset supports scalar int, bool, char, and string values,

local variables, up to six scalar function parameters, user-defined function

calls, integer arithmetic and comparisons, if/else, loops, break,

continue, return, and print. Integer overflow and invalid integer

division terminate the program with status 70.

The experimental backend currently rejects global declarations, arrays,

floats, f-strings, string comparisons, and/or, standard-library or

conversion calls, and multi-file imports. The standard build command continues to use the established C backend.

The separate build-asm command runs NASM and ld for supported programs;

emit-asm writes the assembly source for inspection. The experimental backend

is Linux x86-64 specific and is not supported on macOS or other architectures.

Diagnostic policy

Compiler errors are intended to be stable, user-facing output. Changes should preserve useful codes where practical and include regression tests for source locations and hints. A check failure must not be reported as a native build failure. Code generation is only entered after parsing and semantic analysis report no errors.

Native backend

The v1.0 backend generates C17 and invokes the selected native compiler (`gcc` by default or `clang` when selected). Linux builds use GNU linker RELRO/NOW hardening and `_FORTIFY_SOURCE`; macOS builds omit GNU/Linux-specific flags and use the platform linker. Generated programs are ordinary native executables, not sandboxed. For native builds, generated C is placed in a private temporary directory and removed after the native compiler exits. This avoids clobbering a project file and avoids collisions between concurrent build invocations. `emit-c` writes to the requested path, or `build/ylang-generated.c` if no output path is given.

Scope

This architecture documents the current implementation, not a promise of features outside `docs/language-spec.md`. Arrays, pointers/references, classes, exception syntax, and a finalized memory model are intentionally outside the original 1.0.0 feature set. Named imports are supported by the default C backend; the NASM backend remains a limited experimental subset.

Structs (Experimental)

Source: docs/STRUCTS.md

YLang's current struct milestone supports named fields with explicit types, positional construction, field reads, passing structs to functions, returning structs, and assignment between values of the same struct type.

```
struct Point {
    int x;
    int y;
}

function shift(Point p, int dx) -> Point {
    return Point(p.x + dx, p.y);
}

function main() -> int {
    let Point p = shift(Point(2, 7), 3);
    print(p.x, ",", p.y);
    return 0;
}
```

Current limits

- Declare a struct before using it; its fields must have valid non-void types.
- Construct with positional arguments in declaration order: `Point(2, 7)`.
- Field reads and assignments are supported for fields of a named mutable struct variable, for example `p.x = 10;`. Assignment is type-checked and does not permit assigning a value of a different type.
- No recursive by-value fields, methods, inheritance, generic structs, field defaults, or named constructor arguments.
- The C backend supports structs; NASM parity has not been established.
- This is experimental until the implementation PR and its CI matrix are reviewed and merged.

See YLang 2.0 design (YLANG-2.0-DESIGN.md).

Enums and Matching (Experimental)

Source: docs/ENUMS.md

YLang's current enum milestone supports **unit enums**: named alternatives with no attached payload. Every enum declaration creates a distinct type, even when another enum has the same variants.

```
enum Direction { North, East, South, West }

function is_east(Direction direction) -> bool {
    return direction == Direction.East;
}

function main() -> int {
    let Direction facing = Direction.North;
    match (facing) {
        Direction.North -> { print("north"); }
        Direction.East -> { print("east"); }
        Direction.South -> { print("south"); }
        Direction.West -> { print("west"); }
    }
    return 0;
}
```

Rules

- Declare the enum before using its type or variants.
- Write variants as `EnumName.Variant`.
- Enum values can be stored in variables, passed to functions, returned, and compared with `==` and `!=`.
- Assigning values from different enum types is a type error.
- `match (value)` must include every variant exactly once. Missing variants and duplicate arms are errors.
- Each arm uses `Variant -> statement`; a block is recommended for clarity.
- In this milestone, match values must be simple side-effect-free expressions; store a value in a variable before matching it.
- Enums are represented as integer tags in the C backend. Do not depend on the numeric tag values.
- Payload variants, generic enums, matching bindings, and NASM parity beyond the tested unit-enum subset are not promised as stable.

The syntax and semantics are experimental until the feature PR is reviewed and merged. See YLang 2.0 design (YLANG-2.0-DESIGN.md) for the remaining value-type roadmap.

GUI API (Experimental)

Source: docs/GUI-API-SPEC.md

Status: implemented on `main` as an optional SDL2-backed binding surface. The API below describes the current first milestone; native GUI builds require SDL2 development files, while `check` and ordinary non-GUI builds do not.

This document defines a deliberately small first binding milestone. The contract must be agreed before changing the parser, semantic checker, C generator, and native build driver together.

Goals

- Make a small graphical window usable from ordinary YLang source.
- Preserve the current default build: programs without `gui.*` calls must not need SDL2.
- Keep `check` independent of SDL2 installation; dependency discovery belongs to native `build`.
- Validate argument count and types before code generation.
- Keep the API portable across the project's supported Linux and macOS targets.
- Make unsupported calls fail with YLang diagnostics rather than producing broken C.

Proposed first API

The first milestone intentionally supports one active window per process. It does not expose raw pointers or arbitrary C interop.

Call	Return type	Contract
`gui.init()`	`bool`	Initialize the GUI runtime; false means initialization failed.
`gui.create_window(title, width, height)`	`bool`	Create one window; title is `string`, dimensions are positive `int` values.
`gui.poll()`	`bool`	Process pending events; false means the window was closed or polling failed.
`gui.key_down(key)`	`bool`	Check a named key after polling; supports SDL scancode names such as `Left`, `Space`, and `Escape`.
`gui.mouse_x()`, `gui.mouse_y()`	`int`	Read the mouse position in window coordinates.
`gui.mouse_button_down(button)`	`bool`	Check mouse button state (1 left, 2 middle, 3 right; values 1–5 are accepted).
`gui.ticks_ms()`	`int`	Read SDL's monotonic timer in milliseconds after initialization.
`gui.clear(red, green, blue)`	`bool`	Clear the current frame using RGB channels in the range 0–255.
`gui.draw_rect(x, y, width, height, red, green, blue)`	`bool`	Draw a filled rectangle; dimensions must be positive and color channels 0–255.
`gui.present()`	`bool`	Present the current frame.
`gui.last_error()`	`string`	Return a readable description of the latest GUI runtime error.
`gui.close()`	`void`	Destroy the active window and release its resources. Safe to call after failed creation.
`gui.shutdown()`	`void`	Release initialized GUI runtime resources.

All calls except `gui.close()` and `gui.shutdown()` require successful initialization and, where applicable, a created window. Runtime failures are returned as `false`; the latest human-readable explanation is available through `gui.last_error()`. No implicit numeric conversions are allowed.

Example target syntax

This example shows the intended use of the API and is included in the implementation work.

```
function main() -> int {
    if (not gui.init()) {
        print(gui.last_error());
        return 1;
    }

    if (not gui.create_window("YLang GUI", 800, 500)) {
        print(gui.last_error());
        gui.shutdown();
        return 1;
    }

    loop() {
        if (not gui.poll()) { break; }
        if (not gui.clear(24, 28, 38)) { break; }
        if (not gui.draw_rect(80, 80, 240, 120, 22, 160, 180)) { break; }
        if (not gui.present()) { break; }
    }

    gui.close();
    gui.shutdown();
    return 0;
}
```

The example is available as `examples/gui.yl`. The regression suite checks its types and generated C; an SDL2 runtime smoke test runs when the development package is available.

Compiler and build requirements

1. **Parser:** accept `gui` as a recognized standard-library namespace using the same qualified-call representation as existing namespaces.
2. **Semantic analysis:** define an explicit signature table for every supported call; check arity and each argument's type; assign the documented return type. Unknown `gui.*` names must produce a stable diagnostic.
3. **C generation:** emit calls to a narrow adapter layer. Do not pass arbitrary user-selected C symbols through to generated code.
4. **Native build driver:** detect whether the checked program uses GUI calls. Only then discover SDL2 compiler/linker flags and link the optional runtime. Use argument-vector process execution, not a shell command string.
5. **Portability:** account for the repository-relative runtime source and headers, and report a clear build error if SDL2 development files are missing. The ordinary non-GUI build must continue working without SDL2.
6. **Experimental NASM backend:** reject GUI calls with a clear unsupported-feature diagnostic; do not silently generate invalid assembly.
7. **Documentation:** clearly label this as an optional SDL2 feature and distinguish compile-time checking from dependency-required native building.

Acceptance tests

- `check` accepts a well-typed GUI program even on a machine without SDL2 installed.
- `check` rejects incorrect arity, wrong argument types, unknown GUI names, and invalid types with source-located diagnostics.
- A regular non-GUI program builds and runs with no SDL2 dependency.
- A GUI program builds on Linux and macOS when SDL2 development files are installed.
- A GUI build with missing SDL2 reports the required package/tooling and exits nonzero.
- Runtime initialization/window-creation failure is reportable through `gui.last_error()`.
- The existing C GUI example and all compiler regression tests remain passing.
- The NASM backend rejects GUI programs before assembly generation.

Deliberate non-goals for the first milestone

No widget toolkit, text input, images, audio, a reusable game-loop abstraction, multiple-window handles, custom event objects, arbitrary FFI, or full Pygame-like game framework. Those should build on the first API only after its ownership, error, and lifecycle behavior is tested.

Game2D Library (Experimental)

Source: docs/GAME2D.md

The experimental game2d.yl module is a small, source-level helper layer over YLang's optional SDL2 GUI runtime. It is intended for learning and small 2D prototypes, not as a full game engine.

What is included

- Create and close one window.
- Run a frame loop with event polling, background clear, and present.
- Read keyboard state, mouse position, and mouse buttons.
- Measure elapsed time between frames.
- Draw filled rectangles with RGB colors.
- Test rectangle overlap and point-in-rectangle.
- A complete playable Dodge example that demonstrates movement, timing, collision detection, and score.

This version does not yet provide sprites or image loading, text rendering, audio, animation systems, physics, scene management, or multiple active windows. image.open() opens a file in the operating system's default viewer; it does not draw a sprite inside the game window.

Requirements

- A YLang checkout with the compiler built.
- SDL2 development files and pkg-config.
- A graphical desktop session to run games. CI can check and compile the code, but it cannot interactively test a real game window.

For Fedora:

sudo dnf install SDL2-devel pkgconf-pkg-config

For Debian or Ubuntu:

sudo apt install libsdl2-dev pkg-config

For macOS with Homebrew:

brew install sdl2 pkg-config

Build and play the example

From the repository root, build YLang first, then check and build the sample:

make

make test

make game2d-example

./build/dodge

Controls:

- Left Arrow: move the player left.
- Right Arrow: move the player right.
- Escape: quit.
- Avoid the falling red block. The score increases each time it passes you.

The example writes instructions and the final score to the terminal. The current renderer has no text-drawing API, so the score is not drawn inside the window.

Install the optional runtime and helper module

make install-gui PREFIX="$HOME/.local"

For a user-local installation, make the installed YLang module visible to imports by setting YLANG_PATH to the share directory. For example:

YLANG_PATH="$HOME/.local/share/ylang" ylang build main.yl -o game

Then import the installed module by its path:

from "stdlib/game2d.yl" import game2d_open, game2d_begin_frame, game2d_draw_rect, game2d_end_frame, game2d_close, game2d_key_down, game2d_delta_ms;

The example in this repository uses a relative import because it is intended to run from the checkout.

API reference

Function	Purpose
game2d_open(title, width, height)	Initialize SDL2, create a window, and start the frame timer. Returns bool.
game2d_begin_frame(red, green, blue)	Poll events and clear the frame background. Returns false when the user closes the window or an error occurs.
game2d_draw_rect(x, y, width, height, red, green, blue)	Draw a filled rectangle. Coordinates start at the top-left.
game2d_end_frame()	Present the finished frame.
game2d_key_down(name)	Test whether a key is held, e.g. "Left", "Right", "Space", or "Escape".
game2d_mouse_x() / game2d_mouse_y()	Read the current mouse position.
game2d_mouse_button_down(button)	Test a mouse button; left is 1, middle 2, right 3.
game2d_ticks_ms()	Read elapsed SDL time in milliseconds.
game2d_delta_ms()	Get milliseconds since the previous call. Clamp large values after pauses to avoid big simulation jumps.
game2d_rects_overlap(ax, ay, aw, ah, bx, by, bw, bh)	Axis-aligned rectangle collision test. Rectangles that only touch at an edge do not overlap.
game2d_point_in_rect(px, py, rx, ry, width, height)	Check whether a point is inside a rectangle; the right and bottom edges are excluded.
game2d_last_error()	Return the last GUI/runtime error message.
game2d_close()	Close the window and shut down SDL2. Call it before returning from the game.

RGB channels must be between 0 and 255, and rectangle widths and heights must be positive. Always exit the loop cleanly and call game2d_close() so SDL resources are released.

Example API pattern

from "stdlib/game2d.yl" import game2d_open, game2d_begin_frame, game2d_draw_rect, game2d_end_frame, game2d_close;

function main() -> int {

if (not game2d_open("My game", 640, 400)) {

return 1;

}

loop() {

if (not game2d_begin_frame(20, 24, 35)) {

break;

}

```
if (not game2d_draw_rect(80, 80, 40, 40, 80, 190, 220)) {
break;
}
if (not game2d_end_frame()) {
break;
}
}
game2d_close();
return 0;
}
```

The API and helper implementation are experimental and may change before the stable YLang 2.0 release.

NASM Backend Roadmap

Source: docs/BACKEND-ROADMAP.md

Current state

- YLang's compiler front end and type checker are written in C17.
- The default `ylang build` backend emits C, then invokes GCC or Clang to produce an executable. This is the broadly supported path for the beta.
- The experimental NASM backend emits Linux x86-64 assembly for a restricted subset and links with NASM plus `ld`. It does not support every feature accepted by the C backend.
- Programs requiring the optional SDL2 runtime need the documented native dependencies; the NASM backend rejects GUI calls explicitly.

This means **YLang programs do not yet have a complete, general-purpose native backend independent of GCC/Clang**. The NASM subset is not a drop-in replacement for the C backend.

Design principles

1. **One front end.** Parsing, name resolution, type checking, and diagnostics stay shared. Do not implement language semantics separately inside each backend.
2. **Typed intermediate representation first.** Before expanding the NASM subset or starting another backend, introduce a documented, backend-neutral representation that preserves resolved types, control flow, imports, and source locations.
3. **Declare capabilities.** Each backend must have a tested capability matrix. Unsupported language constructs must cause explicit diagnostics before emission; never silently emit a different behavior or fall back without telling the user.
4. **C remains the reference path for broad coverage.** The C backend stays the default until an alternative covers the required language subset and passes the same behavior tests.
5. **Differential testing is mandatory.** For every construct supported by more than one backend, compile the same source with each backend and compare stdout, stderr/runtime-error classification, and exit status. Keep small regression fixtures for arithmetic, control flow, enum matching, and imports.
6. **Fuzz the shared front end.** Keep lexer/parser unit tests and run the libFuzzer harness under AddressSanitizer and UndefinedBehaviorSanitizer in CI where toolchain support is available.

Proposed milestones

B0 — 2.0 beta

- Keep C/GCC/Clang as the default supported build path.
- Keep NASM explicitly experimental and limited to its documented Linux x86-64 subset.
- Run differential parity tests on the common subset; report skips on unsupported platforms instead of implying parity was tested.

B1 — Backend-neutral typed IR

- Specify expression, statement, type, control-flow, module/import, and source-location representations.
- Add IR invariants and tests independent of any target backend.
- Move code generation gradually behind a backend interface without duplicating semantic checks.

B2 — Native backend proof of concept

Evaluate two options with a small, measurable prototype:

- **LLVM IR/object output**, for mature optimization and target support, while explicitly accounting for LLVM toolchain size and deployment costs.
- **Direct machine-code or assembly generation**, if a small dependency footprint is more important, with target-specific code isolated from the language semantics.

The decision should be based on executable correctness, portability, debug information, compile time, binary size, maintenance cost, and cross-backend test coverage—not on a target date or a single benchmark.

B3 — Expand only behind parity gates

- Extend a backend only when the IR and tests express the feature fully.
- Require parity fixtures for the feature's supported subset and compile-fail diagnostics for unsupported combinations.
- Keep runtime/standard-library dependencies explicit and versioned.

B4 — Reassess the GCC/Clang requirement

Only advertise a general independent backend after it can build and run the supported language suite on every claimed target from a clean checkout. Building YLang's compiler itself may still require a C toolchain unless a separate bootstrap path is implemented.

Out of scope

This is a roadmap, not a promise that LLVM, a direct machine-code backend, Windows support, or a self-hosting compiler will ship in the 2.0 beta. Do not delay the beta for those future milestones, but do not imply that the current NASM subset already removes the C compiler requirement for general YLang programs.

Windows Setup (Experimental)

Source: docs/WINDOWS.md

YLang's native Windows work targets 64-bit Windows with the MinGW-w64 toolchain. The Windows CI smoke job builds the compiler, builds the optional SDL2 GUI C runtime, compiles and runs a simple native program, verifies relative multi-file imports, and tests standard input.

This is the first portability milestone, not a claim that every YLang feature has full Windows parity. NASM output remains Linux x86-64-only. MSVC is not a supported build toolchain for this milestone.

1. Install MSYS2 and the toolchain

Install MSYS2 from https://www.msys2.org/, then open the **MSYS2 MINGW64** terminal (not the plain MSYS terminal).

Update the package database and install the tools:

pacman -Syu

pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-SDL2 mingw-w64-x86_64-pkgconf make python

Ensure the MinGW64 bin directory is on PATH when using YLang from a Windows terminal:

C:\msys64\mingw64\bin

The exact directory can differ if MSYS2 was installed somewhere other than C:\msys64.

2. Build the compiler

Clone the repository from the MINGW64 terminal and build:

git clone https://github.com/Yassine-Jemi01/ylang.git

cd ylang

make clean all CC=gcc

./build/ylang.exe --version

Run the regression suite where the command dependencies are available:

make test CC=gcc

Windows CI currently runs a focused native smoke suite rather than claiming that every POSIX-only shell integration test has been ported.

3. Compile a native Windows program

Create hello.yl:

function main() -> int {

print("Hello, Windows");

print("Answer:", 40 + 2);

return 0;

}

Then:

./build/ylang.exe check hello.yl

./build/ylang.exe build hello.yl -o hello.exe

./hello.exe

The default C backend generates C and invokes GCC or Clang. Use a compiler executable that is available in the environment's PATH.

4. Multi-file imports

Relative imports work from the importing source file:

from "./math.yl" import answer;

YLang treats Windows paths case-insensitively when deduplicating resolved modules. The module search environment variable uses Windows' semicolon-separated path-list convention:

In the MSYS2 MINGW64 shell, use forward slashes in drive paths:

export YLANG_PATH='C:/dev/ylang-libs;D:/shared/ylang'

In PowerShell, use:

$env:YLANG_PATH = "C:/dev/ylang-libs;D:/shared/ylang"

5. Optional GUI and Game2D

The compiler can validate GUI calls without SDL2, but linking and running GUI/game programs requires SDL2 development files and pkg-config. The MINGW64 package command above installs both.

The initial GUI runtime supports one active window, event polling, keyboard/mouse state, timing, background clearing, and filled rectangles. Game2D is intended for simple 2D prototypes and examples. Sprites, audio, in-window text rendering, scene management, and a full physics engine are not part of this milestone.

Current limitations

- Windows support is experimental and the CI smoke tests cover only a focused path through the compiler/runtime.
- MSVC is not supported by this build recipe; use MinGW-w64 GCC.
- The experimental NASM backend is Linux x86-64-only and does not compile all language features.
- Some developer tests and installation scripts still need platform-specific follow-up.
- Installing YLang system-wide and packaging all optional runtime dependencies on Windows is still future work.

When reporting a Windows issue, include the Windows version, MINGW64 package/toolchain versions, exact command, source files, full output, and whether the problem affects check, generated C compilation, or execution.

YSPM Linux Packages

Source: docs/YSPM.md

YSPM is a Linux **system package manager**. It installs native system packages in the `.yspkg` format. It is not a package manager for importing `.yl` modules into an individual YLang project.

YLang's release pipeline can build a native `.yspkg` so Linux users of a compatible YSPM system can install the compiler and its source-level standard library through YSPM. This is an optional Linux distribution path; normal source builds remain available on Linux/macOS, and native Windows follows the MinGW-w64 guide.

Build a .yspkg locally

Requirements:

- Linux x86-64 with a C compiler and GNU Make.
- Go 1.23 or newer to build YSPM from source.
- A YSPM version that supports the native .yspkg build format and the `yspm build` command.

Build YSPM and then the YLang package from the repository root:

From a workspace with sibling ylang and yspm directories:

cd ..

git clone https://github.com/Yassine-Jemi01/yspm.git

cd yspm

go build -o yspm ./cmd/yspm

cd ../ylang

YSPM_BIN="../yspm/yspm" bash packaging/yspm/build-package.sh

The script builds YLang for `/usr`, stages the compiler, .yl standard-library files, GUI runtime sources, public GUI header, license, and available documentation, then asks YSPM to produce:

build/ylang-yspm/ylang-<version>-x86_64.yspkg

The package builder defaults to ABI `yspm-abi-1` and architecture `x86_64`. Override these only to match the target YSPM repository/system. The script defaults to static linking for the compiler executable to reduce undeclared runtime shared-library requirements; set `YLANG_PACKAGE_LDFLAGS` only when intentionally building a different variant.

Install a published package

When a .yspkg asset is published for a YLang release, download that asset and install it with YSPM:

sudo yspm install ./ylang-<version>-x86_64.yspkg

The exact package filename depends on the release version. The package can also be built locally using the command above.

The system package installs the `ylang` executable in `/usr/bin`, source modules in `/usr/share/ylang/stdlib`, the optional GUI runtime source, the GUI public header, and documentation under `/usr/share/doc/ylang`.

For named imports of installed standard-library source modules, add the shared standard-library directory to YLANG_PATH when needed:

export YLANG_PATH=/usr/share/ylang/stdlib

Important boundaries

- YSPM manages Linux **system packages**, not project-local YLang module dependencies.

- The .yspkg is a Linux x86-64 package intended for a compatible YSPM ABI. It is not a macOS or Windows installer.
- The package contains the YLang compiler and source modules, but does not automatically install GCC/Clang, GNU Make, SDL2 development files, or pkg-config. `ylang build` compiles generated C and therefore requires GCC or Clang on the host. GUI/game builds additionally require SDL2 development dependencies and pkg-config.
- The YSPM repository catalog and release signing are managed in the YSPM repository. Publishing a .yspkg as a YLang release asset does not automatically add it to every YSPM repository index.
- A YSPM package is not considered published until the corresponding release asset has been built and attached successfully by CI.

Compiler Error Codes

Source: docs/error-codes.md

YLang diagnostics include stable codes to help users search documentation and report bugs. Wording can improve between releases; documented meanings should remain stable.

Lexing and parsing

Code	Meaning	What to do
E1003	An expression was expected but the current token cannot start one.	Check the preceding operator and use a supported expression.
E1016	Unknown standard-library namespace.	Use a documented namespace such as `math`, `string`, `io`, `path`, or `image`.
E1020	Malformed import declaration or path.	Use `from "./module.yl" import name;`.
E1022	Imported source file could not be resolved or read.	Check the path relative to the importing file.
E1023	Circular module dependency.	Break the import cycle.
E1024	Imported function or global does not exist.	Check the target module declarations and spelling.
E1025	Imported name is duplicated, ambiguous, or conflicts with a local declaration.	Import each name once and avoid local name conflicts.
E1026	An imported module defines or imports the program entry point.	Keep `main()` in the root program file.

Types, names, and expressions

Code	Meaning	What to do
E2002	Unknown type name.	Use a type listed in the language specification.
E2004	Duplicate name in one scope.	Rename or remove the duplicate declaration.
E2010	Integer literal is outside the signed 64-bit range.	Use a representable integer.
E2011	Floating-point literal is invalid or outside the finite range.	Use a finite float literal.
E2012	Character literal is not one byte.	Use one ASCII byte or an escape.
E2020	Unknown variable name.	Declare the variable or correct its spelling.
E2027	Invalid array operation or indexing target.	Index a supported array with an integer index.
E2029	Unsupported array use in a function call or other context.	See array limitations in the language specification.
E2040	Unknown function name.	Check spelling or define the function.
E2041	Wrong number of function or math arguments.	Match the function's documented arity.
E2042	Math function argument is not a float.	Use a float or explicitly call `to_float(int_value)`.
E2043	Wrong number of standard-library or conversion arguments.	Match the function's documented arity.
E2044	A string standard-library function received a non-string argument.	Pass strings.
E2045	An explicit conversion received an unsupported source type.	Check the conversion function's accepted input type.

Runtime arithmetic and bounds failures currently use descriptive text rather than compiler error codes. Runtime source line and function attribution are not yet implemented.

YLang 2.0.0 Stable Release Notes

Source: docs/CHANGELOG-2.0.0.md

YLang 2.0.0 establishes the stable compatibility contract for the documented core language and the default C-backend compiler workflow. This release keeps the language's explicit typing, source-located diagnostics, and inspectable C-generation pipeline as the supported foundation.

Important compatibility boundary: not every component introduced during the 2.0 development cycle is stable. The experimental subsystems listed below may change in future releases and are explicitly outside the stable-core compatibility guarantee.

Stable core

- Explicit declarations, constants, fixed-size homogeneous arrays, scalar types, functions, and structured control flow.
- Explicit conversions and type checking, initialization checks, clear diagnostics, and safe rejection of unsupported syntax.
- The default `ylang build` path: validate `.yl` source, generate C, and invoke GCC or Clang for a native executable.
- Core strings, formatted strings, standard input, text file I/O, math helpers, path utilities, and checked runtime behavior documented by the core specification.
- The `check`, `build`, `emit-c`, and `fix` command-line workflows, with the limitations of each command documented in the README.

The stable language contract is defined in the core language specification (language-spec.md). Where this release notes page and the reference differ, the specification and the explicit experimental labels in the relevant feature guide define the supported behavior.

Experimental extensions

These features are available for evaluation, but are not covered by the stable-core compatibility promise:

- **Named imports and multi-file compilation.** No package registry, dependency solver, import aliases, or qualified module namespaces are promised.
- **Structs and enums.** Several advanced forms remain unsupported; consult the struct reference (STRUCTS.md) and the enum reference (ENUMS.md).
- **GUI and Game2D.** The optional SDL2-backed APIs are not a mature general-purpose game engine. Interactive window behavior requires a graphical session.
- **NASM backend.** Restricted to a documented Linux x86-64 subset; the C backend remains the default.
- **Native Windows support.** Experimental 64-bit MinGW-w64 path with focused smoke tests; MSVC and full feature parity are not claimed.
- **YSPM integration.** Optional Linux x86-64 `.yspkg` delivery for compatible YSPM systems. It is not a Windows/macOS installer and does not supply GCC/Clang or SDL2 development dependencies.

Not implemented in 2.0.0

- `Option<T>` and `Result<T, E>` are not implemented; their design document is a proposal only.
- Classes/OOP, methods on structs, inheritance, generics, and a complete LLVM or independent native backend are not part of this release.
- Arrays cannot be returned or nested, and `char` is one byte rather than a Unicode scalar value.
- YLang does not claim Rust-level memory safety, sandboxing, or suitability for processing hostile source as a hardened security boundary.

Build from source

Requirements are GCC or Clang with C17 support and GNU Make (`gmake` on macOS). For ordinary command-line programs, SDL2 is optional. GUI/Game2D builds additionally need SDL2 development files and `pkg-config`.

```
git clone https://github.com/Yassine-Jemi01/ylang.git
cd ylang
make
make test
./build/ylang --version
```

On macOS, use `gmake` for these build and test commands. Native Windows users should follow the Windows guide (WINDOWS.md) and treat that path as experimental.

Linux YSPM package

The stable release workflow builds the Linux x86-64 `.yspkg` from the exact tested release commit, inspects its contents, runs the packaged compiler, and attaches it only after every stable-release gate passes. Install it only on a compatible YSPM system. Consult the YSPM integration guide (YSPM.md) for ABI, filesystem, and dependency details.

Upgrade and bug reports

Projects moving from YLang 1.1.0 should first run their test suite with 2.0.0 and review experimental feature notes if using imports, structs, enums, GUI/Game2D, NASM, Windows, or YSPM integration.

When reporting a bug, include the exact version and commit, operating system, compiler, command line, a minimal reproducer, expected result, and actual result.

Option and Result Design Proposal

Source: docs/OPTION-RESULT-DESIGN.md

Status: design proposal only. This document does not claim that `Option` or `Result` are implemented.

Goals

- Represent absence and recoverable failure without magic sentinel values.
- Keep YLang's explicit typing rule: no implicit conversion between `int`, `float`, `string`, or other types.
- Make failure paths visible to the type checker.
- Keep the stable 1.1.x language behavior unchanged until the feature is implemented and tested.

Recommended first implementation: tagged values, not generics

The current type system does not yet provide general type parameters. Do not pretend that `Option<T>` or `Result<T, E>` work until generic types, payload layout, and code generation are implemented.

For an incremental milestone, introduce compiler-internal tagged values with concrete payload types, then generalize only after the type representation can express type parameters safely. The surface syntax should be finalized before code changes.

Conceptual semantics:

- `Option<T>`: either `Some(value)` or `None`.
- `Result<T, E>`: either `Ok(value)` or `Err(error)`.
- Matching must be exhaustive, like the existing unit-enum `match` checks.
- Accessing a payload is legal only after the checker proves which variant is active.
- No implicit conversion is performed for payloads or errors.

Proposed user-facing examples (illustrative, not currently compilable)

```
// Illustrative syntax only; do not use as a working example yet.
let Option<int> maybe_port = Some(8080);

match maybe_port {
    Some(port) => { print(port); }
    None => { print("no port"); }
}
```

```
// Illustrative syntax only; do not use as a working example yet.
let Result<int, string> parsed = parse_port("8080");

match parsed {
    Ok(port) => { print(port); }
    Err(message) => { print(message); }
}
```

Implementation sequence

1. **Type representation**: define type identity for payload-bearing variants and ensure incompatible payload types cannot be mixed.
2. **Parser and AST**: parse generic type arguments and variant payload patterns without changing existing enum parsing.
3. **Semantic analysis**: validate constructors, payload types, duplicate arms, exhaustive matching, and payload binding scope.
4. **C backend**: emit a tagged representation with explicit payload storage and safe access only in matching arms.

5. **Assembly backend**: keep the feature disabled for NASM until parity tests exist; do not silently route it through an unverified fallback.
6. **Tests**: cover `Some`/`None`, `Ok`/`Err`, nested payload types, wrong payloads, duplicate/missing arms, and use of payloads outside a matching arm.
7. **Docs and release gate**: label the feature experimental until Linux GCC, Linux Clang, macOS Clang, sanitizers, and backend parity tests pass.

Non-goals for the first milestone

- General user-defined generics.
- Implicit conversions or exception-like hidden control flow.
- Assuming all errors are strings.
- Claiming full assembly-backend support before executable tests pass.

Acceptance criteria

- Stable 1.1.x examples remain unchanged.
- Every invalid payload construction is rejected with a targeted diagnostic.
- Exhaustiveness and payload scope are checked at compile time.
- The generated program has deterministic behavior for every variant.
- CI proves the same observable behavior on supported C compilers.